Rapport IoT Firmware Patch Diffing & CVE Correlation

IoT Firmware Patch Diffing & CVE Correlation

Auteur : **Amadou Tidiane Anne**
Date : Février 2026
Contexte : Embedded Security Project
Stack : Python · Ghidra · Binwalk · NVD API · ARM ASM

Résumé : Ce rapport documente la conception et l'implémentation d'un outil automatisé de patch diffing sur firmwares IoT. L'outil compare deux versions d'un firmware binaire, identifie les fonctions modifiées via Ghidra scripting, et corrèle ces modifications avec les CVE publiés par la National Vulnerability Database (NVD). Cible : firmwares OpenWRT sur architecture MIPS/ARM (TP-Link WDR4300 v1).

Contents

1 Introduction et Contexte **3**
1.1 Problématique . . . 3
1.2 Technique utilisée par l'industrie . . . 3
1.3 Architecture de l'outil . . . 3

2 Environnement et Préparation **3**
2.1 Stack technique . . . 3
2.2 Firmwares cibles . . . 3

3 Phase 1 – Extraction des Firmwares **4**
3.1 Analyse Binwalk . . . 4
3.2 Structure du filesystem extrait . . . 4
3.3 Identification des vecteurs d'attaque potentiels . . . 5
3.4 Identification des binaires modifiés . . . 5

4 Phase 2 – Analyse Ghidra et Hashing **5**
4.1 Script GhidraScript – Extraction des fonctions . . . 6
4.2 Procédure d'import dans Ghidra . . . 6
4.3 Format de sortie JSON . . . 7
4.4 Résultats d'extraction – libuclient.so . . . 7
4.5 Bug identifié – Doublons de projet Ghidra . . . 7

5 Phase 3 – Moteur de Diffing **8**
5.1 Algorithme de comparaison . . . 8
5.2 Résultats du diff – État actuel . . . 9

6 Phase 4 – Corrélation CVE **9**
6.1 CVE identifiés – OpenWRT 22.03.3 vers 22.03.5 . . . 9

6.2 Corrélation confirmée – CVE-2023-24182 10
6.3 Corrélation CVE-2023-0464 / CVE-2023-0465 – En attente 10

7 Résultats et Analyse **10**
7.1 Synthèse des résultats obtenus 10
7.2 Précision de l'outil 11
7.3 Bilan technique du diff engine 11

8 Conclusion **11**

Introduction et Contexte

Problématique

Lorsqu'un fabricant de matériel IoT publie une mise à jour de firmware, il corrige souvent des vulnérabilités sans documenter précisément les fonctions affectées. Cette opacité pose deux problèmes :

- Les administrateurs système ne savent pas ce qui a réellement été corrigé
- Les chercheurs en sécurité ne peuvent pas vérifier l'efficacité du patch

Le **patch diffing** est la technique qui permet de répondre à cette question en comparant directement les binaires des deux versions de firmware.

Technique utilisée par l'industrie

Cette approche est utilisée par :

- **Google Project Zero** – pour analyser les patches Microsoft et Apple
- **Quarkslab** – pour la recherche de vulnérabilités firmware
- **CERT/CC** – pour valider les correctifs de sécurité ICS/SCADA

Architecture de l'outil

```
Firmware v1 (bin) -> Binwalk -> SquashFS -> Ghidra Script -> hash_v1.json -.
Firmware v2 (bin) -> Binwalk -> SquashFS -> Ghidra Script -> hash_v2.json -+
+-> diff_engine.py -> modified_functions.json
+-> nvd_correlator.py -> rapport_final.html
```

Environnement et Préparation

Stack technique

Outil	Rôle
Binwalk	Extraction et analyse des couches du firmware binaire
Ghidra	Désassemblage et analyse statique des binaires extraits
GhidraScript	API Python pour automatiser l'analyse Ghidra
capstone	Désassembleur Python pour analyse des opcodes
NVD API	Base de données nationale des vulnérabilités (NIST)
Python 3	Orchestration, diffing, reporting

Firmwares cibles

Firmwares **OpenWRT** – choisis pour trois raisons :

- Open source : résultats vérifiables contre le code source
- CVE publiés et documentés : corrélation possible
- Architecture MIPS/ARM : représentatif du parc IoT réel

Version	Fichier
22.03.3	`openwrt-22.03.3-ramips-mt7621-tplink_tl-wdr4300-v1-squashfs-sysupgrade.bin`
22.03.5	`openwrt-22.03.5-ramips-mt7621-tplink_tl-wdr4300-v1-squashfs-sysupgrade.bin`

Phase 1 – Extraction des Firmwares

Analyse Binwalk

L'analyse initiale des firmwares via `binwalk` a permis de confirmer la structure des images et de localiser les offsets nécessaires à l'extraction du système de fichiers. Les deux versions partagent une architecture identique mais présentent des variations de taille au niveau du noyau et du rootfs.

Caractéristique	Version 22.03.3	Version 22.03.5
Noyau Linux	5.10.161 (03/01/2023)	5.10.176 (27/04/2023)
Compression Kernel	LZMA	LZMA
Offset SquashFS (Hex)	`0x296805`	`0x2989D7`
Architecture	MIPS32 (Little Endian)	MIPS32 (Little Endian)
Taille SquashFS	3 899 142 octets	3 904 878 octets

Table 1: Comparaison des métadonnées binaires extraites par Binwalk

Listing 1: Sortie Binwalk pour la version vulnérable (v33)

```
DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0x0             uImage firmware image, header size: 64 bytes,
                              data size: 2713541 bytes, compression: lzma,
                              CPU: MIPS32, OS: Linux, image type: OS Kernel
2713605       0x296805        SquashFS file system, little endian, v4.0,
                              compression: xz, block size: 262144,
                              image size: 3899142 bytes
```

Observation technique : Le passage du noyau `5.10.161` au `5.10.176` et l'augmentation du nombre d'inodes (de 1366 à 1368) dans le système SquashFS confirment l'intégration de correctifs de sécurité et de mises à jour de drivers entre les deux versions.

Structure du filesystem extrait

L'extraction du système de fichiers SquashFS a permis de reconstituer l'arborescence complète du système d'exploitation OpenWRT (v22.03.3).

Listing 2: Arborescence simplifiée du rootfs (v33)

```
extracted/v33_rootfs/
|-- bin/                  # Executables essentiels (Busybox, ubus)
|-- etc/                  # Fichiers de configuration
|-- lib/                  # Bibliotheques partagees (libc.so, libcrypto.so)
|   |-- ld-musl-mipsel-sf.so.1 -> libc.so
```

```
|    `-- libustream-ssl.so
|-- sbin/                  # Binaires systeme (procd, ubusd, sysupgrade)
|-- usr/
|    |-- bin/              # Utilitaires utilisateur
|    |-- lib/              # Bibliotheques additionnelles (libwolfssl.so)
|    `-- sbin/             # Services reseau (dnsmasq, dropbear, uhttpd)
`-- www/                   # Interface Web LuCI
```

Identification des vecteurs d'attaque potentiels

Binaire	Chemin	Rôle / Vecteur d'exposition
`uhttpd`	`/usr/sbin/uhttpd`	Serveur Web principal (Interface d'administration)
`wpad`	`/usr/sbin/wpad`	Gestionnaire WPA/WPA2/TLS (WiFi)
`libc.so`	`/lib/libc.so`	Bibliothèque C (Point de passage obligé des syscalls)
`luci`	`/www/cgi-bin/luci`	Point d'entrée de l'API de gestion Web
`dropbear`	`/usr/sbin/dropbear`	Serveur SSH (Accès distant)

Table 2: Binaires critiques identifiés pour l'analyse comparative

Focus Technique : Le binaire `uhttpd` a été sélectionné comme cible prioritaire initiale. L'arborescence confirme la présence de sa configuration dans `/etc/config/uhttpd` et de ses scripts d'initialisation dans `/etc/init.d/uhttpd`.

Identification des binaires modifiés

Un diff complet par hash SHA-256 sur l'ensemble des fichiers des deux rootfs a été réalisé. Parmi les 150+ fichiers modifiés, les binaires ELF identifiés comme cibles prioritaires sont :

Binaire	Modifié	Raison probable
`/lib/libc.so`	Oui	CVE-2023-0464 / CVE-2023-0465 (OpenSSL)
`/usr/sbin/wpad`	Oui	Utilise OpenSSL (WPA2-Enterprise/TLS)
`/usr/lib/libuclient.so`	Oui	Client HTTP/HTTPS – utilise OpenSSL
`/usr/lib/liblucihttp.so.0.1`	Oui	Interface LuCI
`/usr/lib/rpcd/luci.so`	Oui	RPC daemon LuCI
`/bin/busybox`	Oui	Mise à jour générale
`/usr/sbin/uhttpd`	Non	Hash identique – non affecté
`/usr/sbin/dnsmasq`	Non	Hash identique – non affecté

Observation : `uhttpd` et `dnsmasq` ont des hash SHA-256 identiques entre v33 et v35 – ils n'ont pas été modifiés par ce patch. La cible retenue pour l'analyse Ghidra est `libuclient.so`, dont les hash SHA-256 diffèrent :
`v33: 26e11c0a...acaa76f3`
`v35: 3d86e323...fde03ad8`

Phase 2 – Analyse Ghidra et Hashing

Script GhidraScript – Extraction des fonctions

Le script `GhidraFunctionHasher.py` a été développé pour automatiser l'extraction de métadonnées de chaque fonction identifiée par Ghidra. Il s'exécute depuis la console de scripting Ghidra (`Window > Script Manager`) et exploite l'API Java exposée via Jython.

Listing 3: GhidraFunctionHasher.py – Extraction et hashing des fonctions

```
# @category: Vulnerability Analysis
# @author: Amadou Tidiane Anne

import hashlib, json, os
from ghidra.program.model.listing import CodeUnit
from ghidra.util.task import ConsoleTaskMonitor

def get_opcode_hash(func):
    opcodes = ""
    listing = currentProgram.getListing()
    addr_set = func.getBody()
    code_units = listing.getCodeUnits(addr_set, True)
    for cu in code_units:
        if isinstance(cu, ghidra.program.model.listing.Instruction):
            opcodes += cu.getMnemonicString()
    if not opcodes:
        return "no_instructions_found"
    return hashlib.sha256(opcodes.encode()).hexdigest()

def run_analysis():
    results = []
    fm = currentProgram.getFunctionManager()
    for func in fm.getFunctions(True):
        results.append({
            "name":        func.getName(),
            "address":     func.getEntryPoint().toString(),
            "size":        func.getBody().getNumAddresses(),
            "opcode_hash": get_opcode_hash(func),
            "calls": [f.getName() for f in
                      func.getCalledFunctions(ConsoleTaskMonitor())]
        })
    output_path = os.path.join(os.path.expanduser("~"),
                  currentProgram.getName() + "_analysis.json")
    with open(output_path, "w") as f:
        json.dump(results, f, indent=4)
```

Principe du hashing par opcodes : Plutôt que de comparer les adresses mémoire (qui changent à chaque compilation), le script extrait les *mnémoniques* de chaque instruction (ex : `sw`, `jalr`, `lw`) et calcule leur hash SHA-256 concaténé. Cette approche est **insensible aux rebasages** – seul un changement de logique fonctionnelle modifie le hash.

Procédure d'import dans Ghidra

L'import des binaires MIPS dans Ghidra nécessite une configuration spécifique pour assurer une analyse correcte :

1. **Format :** Executable and Linking Format (ELF)
2. **Langage :** `MIPS:LE:32:default:default` (Little Endian, 32 bits)
3. Activer les analyseurs : `Function ID`, `Demangler GNU`, `MIPS Constant Reference Analyzer`

4. Exécuter le script depuis `Window > Script Manager > Run`

Attention – Import séparé obligatoire : Chaque version du binaire doit être importée comme un *programme distinct* dans Ghidra. Réutiliser le même projet sans recharger le fichier produit deux exports JSON identiques même si les binaires diffèrent (voir Section 4.5).

Format de sortie JSON

Chaque entrée du fichier JSON exporté par le script contient :

Listing 4: Format JSON de sortie – exemple réel extrait de libuclient_v33.json

```
{
  "name":         "uclient_get_addr",
  "address":      "000117a8",
  "size":         64,
  "opcode_hash": "b3f12a9c4d...",
  "calls":        ["inet_ntop", "strcpy", "ntohs"]
}
```

Résultats d'extraction – libuclient.so

Le script a été exécuté sur `libuclient.so` (binaire cible dont le SHA-256 diffère entre les deux versions). Les résultats d'extraction sont les suivants :

Métrique	Version 22.03.3 (v33)	Version 22.03.5 (v35)
Fonctions extraites	81	81
Fonctions avec opcodes	78	78
Fonctions sans instructions	3	3
Taille min. (bytes)	1	1
Taille max. (bytes)	422	422
Taille moyenne (bytes)	49	49

Table 3: Statistiques d'extraction Ghidra – libuclient.so v33 vs v35

Fonctions sans instructions désassemblées : Trois fonctions stub (`__deregister_frame_info`, `__cxa_finalize`, `__register_frame_info`) n'ont pas pu être désassemblées par Ghidra (`opcode_hash = "no_instructions_found"`). Ces fonctions ont une taille de 1 byte – elles correspondent à des entrées de la PLT (Procedure Linkage Table) non résolues statiquement.

Bug identifié – Doublons de projet Ghidra

Lors de l'analyse comparative, une anomalie critique a été détectée : les deux fichiers JSON exportés (`libuclient_v33.json` et `libuclient_v35.json`) produisent **zéro différence** malgré des SHA-256 distincts sur les binaires sources.

Diagnostic : L'investigation programmatique a révélé que les 81 hashes d'opcodes sont strictement identiques entre les deux exports :

Listing 5: Vérification de l'identité des hashes entre les deux exports

```
$ python3 -c "
import json
v33 = {f['name']:f for f in json.load(open('libuclient_v33.json'))}
```

```
v35 = {f['name']:f for f in json.load(open('libuclient_v35.json'))}
same = all(v33[n]['opcode_hash'] == v35[n]['opcode_hash'] for n in
    v33)
print('Tous les hashes identiques:', same)  # True
"
Tous les hashes identiques: True
```

Cause racine : Le script GhidraScript a été exécuté deux fois sur le *même programme chargé* dans Ghidra, au lieu d'être appliqué sur deux programmes distincts (v33 et v35 importés séparément). Ghidra maintient un cache d'analyse par projet – si le fichier source n'est pas rechargé, l'analyse porte sur le même binaire.

Correction à appliquer :

1. Dans Ghidra : `File > New Project` pour chaque version
2. Importer `libuclient_v33.so` dans le projet v33, analyser, exécuter le script
3. Importer `libuclient_v35.so` dans le projet v35, analyser, exécuter le script
4. Vérifier que les deux JSON produits ont des SHA-256 différents avant de lancer le diff

Impact sur les résultats : Les comparaisons de la section suivante s'appuient sur les JSON actuels. Tant que les exports Ghidra corrects ne sont pas produits, les résultats de diffing sur les opcodes restent à confirmer. Les sections CVE (diff `sshkeys.js`, SHA-256 des binaires) ne sont pas affectées par ce bug.

Phase 3 – Moteur de Diffing

Algorithme de comparaison

Le moteur de diffing (`DiffEngine.py`) implémente une comparaison multi-critères entre les deux exports JSON :

Listing 6: DiffEngine.py – Comparaison multi-critères

```
def compare(v33_path, v35_path):
    v33 = {item['name']: item for item in json.load(open(v33_path))}
    v35 = {item['name']: item for item in json.load(open(v35_path))}

    modified, removed = [], []
    added = [name for name in v35 if name not in v33]

    for name, data_v33 in v33.items():
        if name in v35:
            data_v35 = v35[name]
            # Comparaison hash opcodes ET taille
            if (data_v33['opcode_hash'] != data_v35['opcode_hash']
                    or data_v33['size'] != data_v35['size']):
                modified.append((name, data_v33, data_v35))
        else:
            removed.append(name)
    return added, removed, modified
```

La stratégie de comparaison est hiérarchique :

1. **Hash des opcodes (SHA-256)** – détecte tout changement de logique assembleur

2. **Taille en bytes** – détecte les ajouts/suppressions d'instructions même si le hash coïncide par collision
3. **Similarité Jaccard** (`diff_engine.py`) – mesure de similarité fine entre ensembles d'opcodes pour les cas ambigus

La similarité Jaccard est définie par :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

où A et B sont les ensembles de mnémoniques des deux versions d'une fonction. Un seuil de $J < 0.8$ signale une modification significative.

Résultats du diff – État actuel

Les résultats produits par le diff engine sur les exports JSON actuels sont les suivants :

Catégorie	Nombre	Observations
Fonctions ajoutées	0	Aucune nouvelle fonction dans v35
Fonctions supprimées	0	Aucune suppression détectée
Fonctions modifiées (hash)	0	Résultat nul dû au bug Ghidra
Différences d'ordre `calls`	8	Non-déterminisme de l'API Ghidra

Table 4: Résultats du diff engine sur libuclient.so – exports actuels

Analyse des différences d'ordre `calls` : L'investigation a révélé 8 fonctions où la liste `calls` contient les mêmes éléments mais dans un ordre différent entre v33 et v35 :

Listing 7: Exemple de différence d'ordre dans les appels – uclient_get_addr

```
v33: "calls": ["inet_ntop", "strcpy", "ntohs"]
v35: "calls": ["inet_ntop", "ntohs",  "strcpy"]
```

Cette divergence est un **artefact de l'API Ghidra** : `getCalledFunctions()` ne garantit pas d'ordre de retour déterministe. Ce n'est pas un indicateur de modification du code binaire.

Correction dans le diff engine : Comparer les listes `calls` en utilisant `set()` plutôt qu'une comparaison directe de liste pour éviter les faux positifs liés au non-déterminisme de l'API.

Listing 8: Correction recommandée pour la comparaison des calls

```
# Avant (faux positifs possibles)
if data_v33['calls'] != data_v35['calls']:
    ...

# Apres (comparaison insensible a l'ordre)
if set(data_v33['calls']) != set(data_v35['calls']):
    ...
```

Phase 4 – Corrélation CVE

CVE identifiés – OpenWRT 22.03.3 vers 22.03.5

Les CVE patchés dans cette version ont été identifiés via deux sources : les release notes officielles OpenWRT et l'API NVD (NIST).

CVE	CVSS	Type	Description
CVE-2023-0464	7.5	DoS OpenSSL	Excessive Resource Usage Verifying X.509 Policy Constraints
CVE-2023-0465	5.3	OpenSSL	Invalid certificate policies in leaf certificates silently ignored
CVE-2023-24181	5.4	XSS (LuCI)	Reflected XSS via `/openvpn/pageswitch.htm`
CVE-2023-24182	5.4	XSS (LuCI)	Stored XSS via `/system/sshkeys.js`

Corrélation confirmée – CVE-2023-24182

L'analyse diff de `sshkeys.js` a permis d'identifier et de confirmer le patch du CVE-2023-24182. La modification est chirurgicale – une seule ligne changée :

Listing 9: Diff sshkeys.js – CVE-2023-24182

```
# Version 22.03.3 (vulnerable)
- E('pre', delkey)

# Version 22.03.5 (patched)
+ E('pre', [delkey])
```

Analyse de la vulnérabilité :

- `E('pre', delkey)` – la clé SSH est passée directement comme chaîne au DOM LuCI
- Si la clé contient du HTML/JavaScript, le navigateur l'interprète – XSS stocké
- `E('pre', [delkey])` – enveloppé dans un tableau, LuCI le traite comme texte brut
- Un attaquant pouvait injecter : `ssh-rsa AAAA...  <img src=x onerror=alert(1)>`

CVE-2023-24182 : CONFIRMÉ – Patch localisé et analysé dans `/www/luci-static/resources/view/system/sshkeys.js`, ligne 30. Modification minimale mais critique : passage de string brut à tableau pour échappement automatique.

CVE-2023-24181 : Fichier `pageswitch.htm` absent de ce firmware – module OpenVPN non installé sur ce modèle TP-Link WDR4300. CVE non vérifiable sur cette image.

Corrélation CVE-2023-0464 / CVE-2023-0465 – En attente

Ces CVE affectent OpenSSL embarqué dans `libc.so`. La confirmation binaire nécessite des exports Ghidra corrects (deux programmes séparés – voir Section 4.5). Les SHA-256 distincts de `libuclient.so` entre v33 et v35 constituent une présomption forte de modification liée à ces CVE, mais la corrélation au niveau fonction reste à établir.

Les fonctions OpenSSL candidates attendues dans le diff sont : `X509_policy_check()`, `d2i_X509_POLICY_NODE(`
et les routines de validation de chaînes X.509.

Résultats et Analyse

Synthèse des résultats obtenus

Élément analysé	Résultat	Statut
Extraction Binwalk v33 / v35	Succès – offset, taille, architecture	Confirmé
Hash SHA-256 des binaires ELF	6 binaires modifiés identifiés	Confirmé
CVE-2023-24182 (XSS sshkeys.js)	Patch localisé, ligne 30 confirmée	Confirmé
CVE-2023-24181 (XSS OpenVPN)	Module absent du firmware cible	Non applicable
Export Ghidra libuclient.so	Bug : deux exports identiques produits	A corriger
Diff engine opcodes	0 diff (artefact bug Ghidra)	A revalider
CVE-2023-0464 / 0465 (OpenSSL)	Présomption forte, conf. binaire requise	En cours

Table 5: Tableau de bord des résultats – état du projet

Précision de l'outil

Sur la base des résultats obtenus à ce stade :

- **CVE-2023-24182 :** corrélation confirmée à 100% par analyse textuelle directe (fichier JavaScript, pas de désassemblage requis).
- **CVE-2023-24181 :** non applicable sur ce firmware (module manquant), mais le système l'a correctement détecté via `find`.
- **CVE OpenSSL :** corrélation au niveau binaire bloquante par le bug Ghidra – à revalider après correction de la procédure d'import.

La précision cible de 75% sur la corrélation CVE/fonction est atteignable une fois les exports Ghidra corrects produits, au vu de la qualité de l'extraction (81 fonctions, hashes stables, calls correctement répertoriés).

Bilan technique du diff engine

Le moteur de diffing est fonctionnellement correct. Les deux implémentations (`diff_engine.py` et `DiffEngine.py`) produisent des résultats cohérents sur les métadonnées disponibles. Le seul correctif nécessaire est la comparaison insensible à l'ordre des `calls` (utilisation de `set()`).

Conclusion

Ce projet a permis de mettre en place un pipeline complet de patch diffing sur firmwares IoT, depuis l'extraction Binwalk jusqu'à la corrélation CVE. Plusieurs résultats concrets ont été obtenus :

- Le **CVE-2023-24182** a été confirmé et analysé – une modification d'une seule ligne dans `sshkeys.js` constitue le patch complet d'un XSS stocké critique.
- L'**outillage Ghidra** (script d'extraction, moteur de diff, similarité Jaccard) est opérationnel et extensible à tout binaire ELF MIPS/ARM.
- Un **bug de procédure** sur les exports Ghidra a été identifié et diagnostiqué programmatiquement – les exports corrects restent à produire pour finaliser la corrélation des CVE OpenSSL.

Limites de l'outil :

- Les fonctions stub (PLT non résolues) ne sont pas désassemblées par Ghidra en mode statique – une analyse dynamique compléterait ce cas.
- La similarité Jaccard sur les mnémoniques seuls peut produire des faux négatifs si la logique

change mais que le jeu d'instructions reste identique (ex : remplacement d'un `strcpy` par `strncpy`).

- L'outil est actuellement mono-binaire – l'extension à un diff système complet (tous les ELF du rootfs en une passe) améliorerait la couverture.

Améliorations futures :

- Intégrer **capstone** pour un hashing au niveau des opérandes (plus précis que les seuls mnémoniques)
- Ajouter un **mode headless Ghidra** pour automatiser l'export sans interface graphique
- Générer un **rapport HTML interactif** avec liens vers la NVD pour chaque CVE corrélé

Bilan : Malgré le bug d'export Ghidra sur `libuclient.so`, le pipeline global est validé. Le CVE-2023-24182 a été localisé et analysé avec précision. La correction de la procédure Ghidra permettra de finaliser la corrélation des CVE OpenSSL et d'atteindre l'objectif de 75% de précision.

Références

- **NIST National Vulnerability Database** – `https://nvd.nist.gov`
- **OpenWRT Project** – `https://openwrt.org`
- **Ghidra NSA Tool** – `https://ghidra-sre.org`
- **Binwalk Firmware Analysis** – `https://github.com/ReFirmLabs/binwalk`
- **Amadou Tidiane Anne**, *Analyse de la pertinence des métriques système natives pour la détection d'anomalies sous Linux en environnements contraints*, HAL Open Science, 2026. `https://hal.science/hal-05486729v1`